

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 2

Error Analysis Task

COURSE NAME: Deep Learning
FACULTY : Dr.Arul Raj A.M

COURSE CODE: MLA0403

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 30%

Duration: 30 minutes

Marks: 25

Code of Conduct:

I V Yogeswar (192425175) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

QUESTIONS:

Total Marks:25

1. Error Analysis in Maximum Likelihood Estimation (MLE)

A student builds a binary classifier using Maximum Likelihood Estimation. The likelihood function is incorrectly written as the sum of probabilities instead of the product. The model produces poor parameter estimates even with large data.

Tasks:

a) Conceptual Error

The likelihood function should be the **product of probabilities**, not the **sum**. Using the sum gives an incorrect likelihood.

b) Effect on Parameter Estimation

- Produces incorrect parameter estimates.

- The model cannot find the true maximum likelihood.
- Accuracy decreases even with large datasets.

c) Correct Mathematical Formulation

For independent samples:

$$L(\theta) = \prod_{i=1}^n P(x_i | \theta)$$

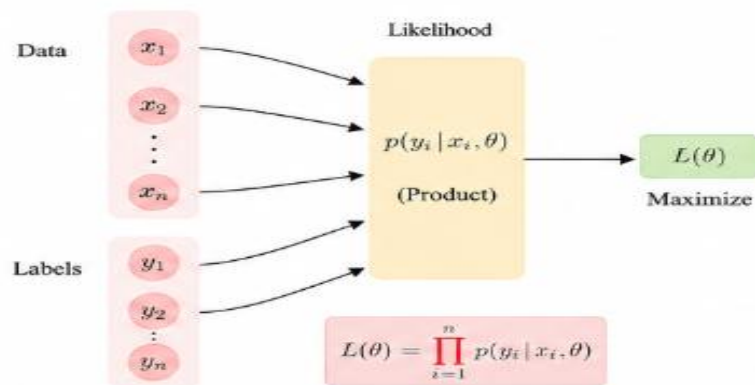
This correctly represents the probability of observing all data samples.

d) Log-Likelihood

Taking the logarithm converts the product into a sum:

$$\log L(\theta) = \sum_{i=1}^n \log P(x_i | \theta)$$

This avoids numerical underflow, simplifies differentiation, and makes optimization easier.



2. Error Diagnosis in Perceptron Learning

While implementing a Perceptron, a student observes that the model fails to converge even after many iterations on a linearly separable dataset.

Tasks:

a) Possible Implementation Errors

- Incorrect weight update rule.
- Wrong learning rate.
- Bias term missing.
- Incorrect prediction function.
- Incorrect stopping condition.

b) Effect of Learning Rate and Weight Updates

- A very high learning rate causes overshooting.

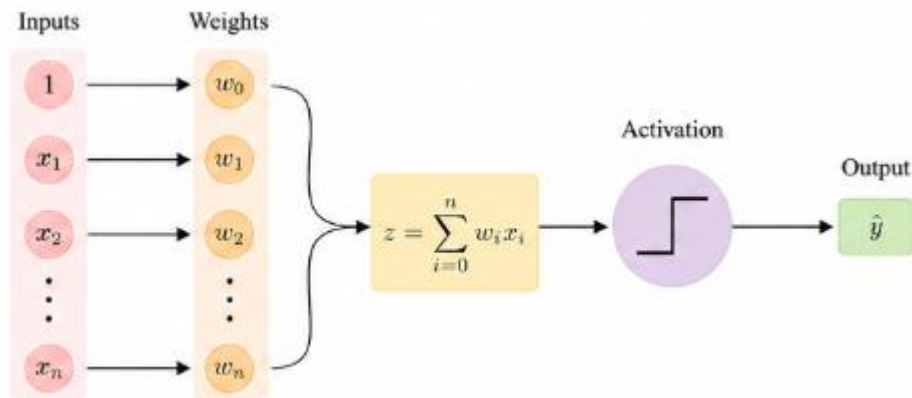
- A very low learning rate slows convergence.
- Incorrect weight updates prevent the model from learning correctly.

c) Dataset-Related Issues

- Data not actually linearly separable.
- Incorrect labels.
- Missing feature scaling.
- Noisy or duplicate samples.

d) Modifications

- Use the correct Perceptron update rule.
- Choose an appropriate learning rate.
- Normalize input features.
- Include the bias term.
- Verify that the dataset is linearly separable.



3. Backpropagation Gradient Error Analysis

A Multilayer Perceptron trained using Backpropagation shows increasing loss during training instead of decreasing.

Tasks:

a) Possible Errors

- Incorrect gradient calculations.
- Wrong derivative of activation function.
- Incorrect chain rule implementation.
- Wrong learning rate.
- Coding bugs in backpropagation.

b) Role of Activation Functions

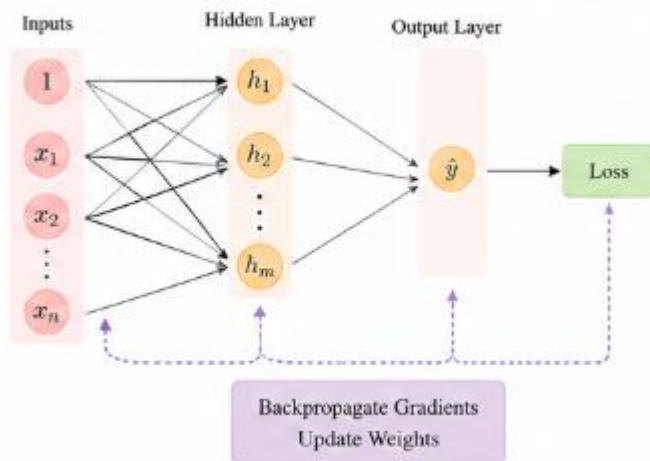
Activation functions introduce non-linearity. Their derivatives determine how gradients flow through the network during backpropagation.

c) Vanishing and Exploding Gradients

- **Vanishing gradients:** Gradients become very small, slowing or stopping learning.
- **Exploding gradients:** Gradients become very large, causing unstable training and increasing loss.

d) Solutions

- Use Xavier or He initialization.
- Apply Batch Normalization.
- Use ReLU activation.
- Apply Gradient Clipping.
- Choose an appropriate learning rate.



4. Gradient Descent vs SGD Error Investigation

A model trained using Gradient Descent converges very slowly, while the same model with Stochastic Gradient Descent shows unstable loss fluctuations.

Tasks:

a) Why Batch Gradient Descent is Slow

- Uses the entire dataset for every update.
- Computationally expensive for large datasets.
- Takes longer to complete each iteration.

b) Cause of SGD Fluctuations

- SGD updates weights using one sample at a time.
- Updates are noisy, causing fluctuations in loss.

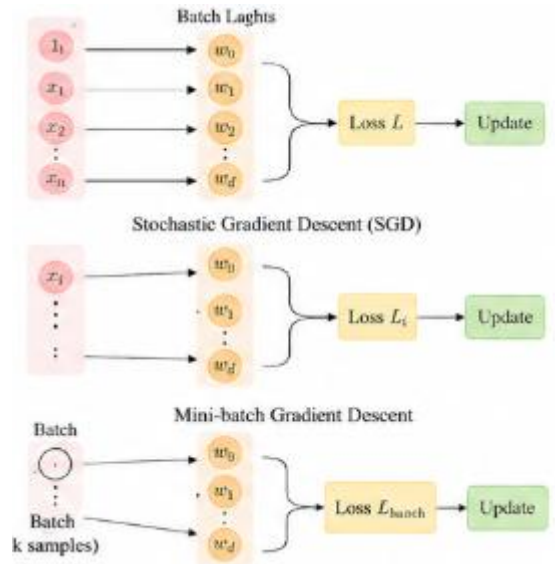
c) Mini-Batch Gradient Descent

- Uses small batches of data.
- Faster than Batch Gradient Descent.
- More stable than SGD.
- Provides a balance between speed and stability.

d) Best Approach

Mini-Batch Gradient Descent is preferred because:

- Faster convergence.
- Less noisy than SGD.
- More computationally efficient than Batch Gradient Descent.
- Better balance between speed and accuracy.



5. Curse of Dimensionality Error Analysis

A machine learning model built using high-dimensional data performs well on training data but poorly on test data due to overfitting, related to the Curse of Dimensionality.

Tasks:

a) How High Dimensionality Increases Error

- Data becomes sparse.
- More parameters increase model complexity.
- The model easily memorizes training data, causing overfitting.

b) Signs of Overfitting

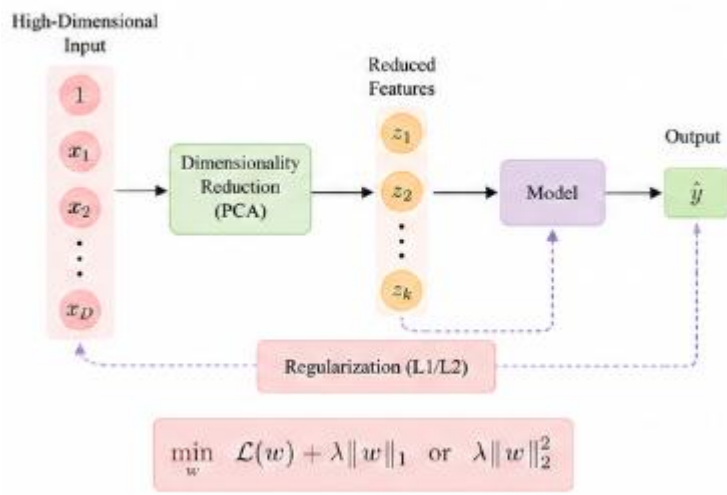
- High training accuracy.
- Low testing accuracy.
- Large gap between training and testing performance.

c) Why Distance-Based Learning Fails

In high dimensions, distances between data points become very similar, making it difficult to distinguish nearest neighbors accurately.

d) Improvement Techniques

- **PCA (Principal Component Analysis):** Reduces feature dimensions.
- **Feature Selection:** Removes irrelevant features.
- **L1/L2 Regularization:** Reduces model complexity.



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts

Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand